

UNIVERSIDADE PAULISTA – UNIP

Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas

Pedro Oliveira Andrade – RA: 4173932

Eduardo Andrade de Castro – RA: 3312782

Matheus Hiroshi Yaneda Ofuro – RA: 9476025

Vinicius Gomes Correa – RA: 6173461

Luigi Mikael Grimm – RA: 8175617

Ivan Rocha Campos Neto – RA: H77FAF6

DOJODOV – PLATAFORMA EDUCACIONAL GAMIFICADA

Projeto Integrado Multidisciplinar III

São Paulo

2026

Pedro Oliveira Andrade – RA: 4173932

Eduardo Andrade de Castro – RA: 3312782

Matheus Hiroshi Yaneda Ofuro – RA: 9476025

Vinicius Gomes Correa – RA: 6173461

Luigi Mikael Grimm – RA: 8175617

Ivan Rocha Campos Neto – RA: H77FAF6

DOJODOV – PLATAFORMA EDUCACIONAL GAMIFICADA

Projeto Integrado Multidisciplinar III para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador(a): Professor(a) da Disciplina PIM III

São Paulo

2026

RESUMO

Este trabalho apresenta o DojoDev, uma plataforma educacional gamificada desenvolvida como projeto prático no âmbito do Projeto Integrado Multidisciplinar III (PIM III) do curso de Análise e Desenvolvimento de Sistemas da UNIP. A plataforma tem como objetivo facilitar o aprendizado de programação por meio de mecânicas de jogos, integrando desafios, sistema de pontuação, ranking e prova de nivelamento. O desenvolvimento contemplou modelagem de banco de dados relacional, implementação do back-end em C# com orientação a objetos, construção do front-end web responsivo, aplicação de conceitos de UX/UI Design, análise de dados e aspectos de comunicação e liderança. Foram elaborados os diagramas UML (casos de uso, classes e sequência) e o modelo entidade-relacionamento do banco de dados. Os resultados demonstram a integração bem-sucedida das disciplinas do semestre em um produto funcional, moderno e acessível.

Palavras-chave: gamificação; plataforma educacional; programação; C#; web responsivo; banco de dados; UML; machine learning.

ABSTRACT

This paper presents DojoDev, a gamified educational platform developed as a practical project within the Integrated Multidisciplinary Project III (PIM III) of the Systems Analysis and Development course at UNIP. The platform aims to facilitate programming learning through game mechanics, integrating challenges, a scoring system, ranking, and a leveling test. The development covered relational database modeling, back-end implementation in C# with object-oriented programming, responsive web front-end construction, UX/UI Design concepts, data analysis and communication/leadership aspects. UML diagrams (use cases, classes, and sequence) and the entity-relationship model were elaborated. The results demonstrate the successful integration of the semester's disciplines into a functional, modern, and accessible product.

Keywords: gamification; educational platform; programming; C#; responsive web; database; UML; machine learning.

LISTA DE FIGURAS

Figura 1 – Diagrama de Casos de Uso (UseCase Diagram)	13
Figura 2 – Diagrama de Classes (visão geral)	14
Figura 3 – Diagrama de Classes (Usuário e Dev)	15
Figura 4 – Diagrama de Classes (Estudante, Curso e ProvaNivelamento)	15
Figura 5 – DER – Diagrama Entidade-Relacionamento	10
Figura 6 – MER – Modelo Entidade-Relacionamento	11
Figura 7 – Diagrama de Sequência: Realizar Prova de Nivelamento	16
Figura 8 – Diagrama de Sequência: Completar Desafio	17

SUMÁRIO

1 INTRODUÇÃO	7
2 DESENVOLVIMENTO	8
2.1 Descrição do Projeto – DojoDev	8
2.2 Modelagem de Banco de Dados	9
2.3 Diagramas UML	13
2.4 Back-End em C# (POO)	18
2.5 Desenvolvimento Web Responsivo	22
2.6 UX e UI Design	24
2.7 Machine Learning e Análise de Dados	25
2.8 Comunicação, Liderança, Negociação e LIBRAS	26
2.9 Integração do Sistema	27
3 CONCLUSÃO	28
REFERÊNCIAS	29

1 INTRODUÇÃO

O aprendizado de programação tem se mostrado cada vez mais relevante no contexto atual, em que a transformação digital impacta todas as áreas profissionais e pessoais. No entanto, uma das principais barreiras para ingressantes na área é a falta de engajamento e a curva de aprendizado acentuada das linguagens de programação. Plataformas gamificadas surgem como uma alternativa eficiente para superar esses desafios, combinando conteúdo técnico com mecânicas de jogos.

Nesse contexto, o presente trabalho apresenta o DojoDev, uma plataforma educacional gamificada com temática ninja/dojo, desenvolvida como projeto prático do PIM III – Projeto Integrado Multidisciplinar do terceiro semestre do curso de Análise e Desenvolvimento de Sistemas (ADS) da Universidade Paulista (UNIP).

O objetivo principal do projeto é desenvolver uma aplicação funcional que integre os conhecimentos das disciplinas do semestre: Modelagem de Banco de Dados e NoSQL, Programação Orientada a Objetos com C#, Desenvolvimento Web Responsivo, UX/UI Design, Machine Learning e Análise de Dados, e Comunicação e Liderança. O sistema oferece trilhas de aprendizado, desafios com pontuação em XP (pontos de experiência), sistema de ranking, prova de nivelamento e painel administrativo.

O trabalho está organizado em três partes principais: introdução, desenvolvimento – onde são apresentadas todas as etapas do projeto – e conclusão, com as considerações finais e perspectivas de evolução da plataforma.

2 DESENVOLVIMENTO

2.1 Descrição do Projeto – DojoDev

O DojoDev é uma plataforma educacional gamificada voltada ao ensino de programação, desenvolvida com design dark de temática ninja/dojo. A plataforma é totalmente responsiva, funcionando adequadamente em dispositivos desktop, tablet e mobile.

O sistema contempla três perfis de usuários distintos:

- Estudante: acesso ao dashboard de aprendizado, trilhas, desafios e ranking;
- Professor (Tutor): painel para criação e gerenciamento de trilhas e conteúdos;
- Administrador (Dev): dashboard administrativo com métricas gerais e gerenciamento de usuários.

As funcionalidades principais incluem:

- Sistema de XP e níveis: o estudante ganha pontos ao completar desafios e sobe de nível conforme acumula experiência;
- Streaks diários: incentivo à construção do hábito de estudo;
- Prova de nivelamento: avalia o conhecimento prévio do estudante e posiciona-o na trilha adequada;
- Ranking: classificação dos estudantes por XP acumulado;
- Chat Sensei: assistente interativo para dúvidas;
- Trilhas de aprendizado: JavaScript Iniciante, HTML & CSS e Python Iniciante.

As tecnologias utilizadas no front-end incluem HTML5, CSS3 (com Flexbox, Grid e Media Queries), JavaScript Vanilla e Tailwind CSS. Os dados são persistidos localmente via localStorage.

2.2 Modelagem de Banco de Dados

2.2.1 Scripts SQL – Criação do Banco de Dados

O banco de dados relacional do sistema foi desenvolvido no MySQL sob o nome SistemaAcademico. O modelo contempla sete entidades principais, com relacionamentos definidos por chaves primárias e estrangeiras.

```
CREATE DATABASE SistemaAcademico;
USE SistemaAcademico;

CREATE TABLE Usuario (
  id_usuario INT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  senha VARCHAR(255) NOT NULL,
  tipo_usuario ENUM('Estudante','Tutor','Dev') NOT NULL
);

CREATE TABLE Estudante (
  id_estudante INT PRIMARY KEY,
  nivel INT DEFAULT 1,
  experiencia_total INT DEFAULT 0,
  ranking_posicao INT,
  FOREIGN KEY (id_estudante) REFERENCES Usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE Tutor (
  id_tutor INT PRIMARY KEY,
  especialidade VARCHAR(100),
  FOREIGN KEY (id_tutor) REFERENCES Usuario(id_usuario) ON DELETE CASCADE
);

CREATE TABLE Curso (
  id_curso INT PRIMARY KEY AUTO_INCREMENT,
  nome_curso VARCHAR(100) NOT NULL,
  area_estudo VARCHAR(50) NOT NULL
);

CREATE TABLE Desafio (
  id_desafio INT PRIMARY KEY AUTO_INCREMENT,
  titulo VARCHAR(100) NOT NULL,
  tipo_desafio VARCHAR(50),
  dificuldade INT,
  recompensa_xp INT NOT NULL,
  id_curso INT,
  id_tutor INT,
  FOREIGN KEY (id_curso) REFERENCES Curso(id_curso),
  FOREIGN KEY (id_tutor) REFERENCES Tutor(id_tutor)
);

CREATE TABLE ProvaNivelamento (
  id_prova INT PRIMARY KEY AUTO_INCREMENT,
  dificuldade_inicial INT,
  resultado_nivel INT,
  id_estudante INT,
  FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante)
);

CREATE TABLE ProgressoEstudante (
  id_progresso INT PRIMARY KEY AUTO_INCREMENT,
  id_estudante INT,
  id_desafio INT,
```

```

data_conclusao DATETIME DEFAULT CURRENT_TIMESTAMP,
xp_ganho      INT,
FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante),
FOREIGN KEY (id_desafio)   REFERENCES Desafio(id_desafio)
);

CREATE TABLE Ranking (
  id_ranking      INT PRIMARY KEY AUTO_INCREMENT,
  periodo         VARCHAR(20),
  id_estudante    INT,
  posicao          INT,
  FOREIGN KEY (id_estudante) REFERENCES Estudante(id_estudante)
);

```

2.2.2 DER – Diagrama Entidade-Relacionamento

O DER abaixo representa graficamente as tabelas do sistema e seus relacionamentos no MySQL Workbench:

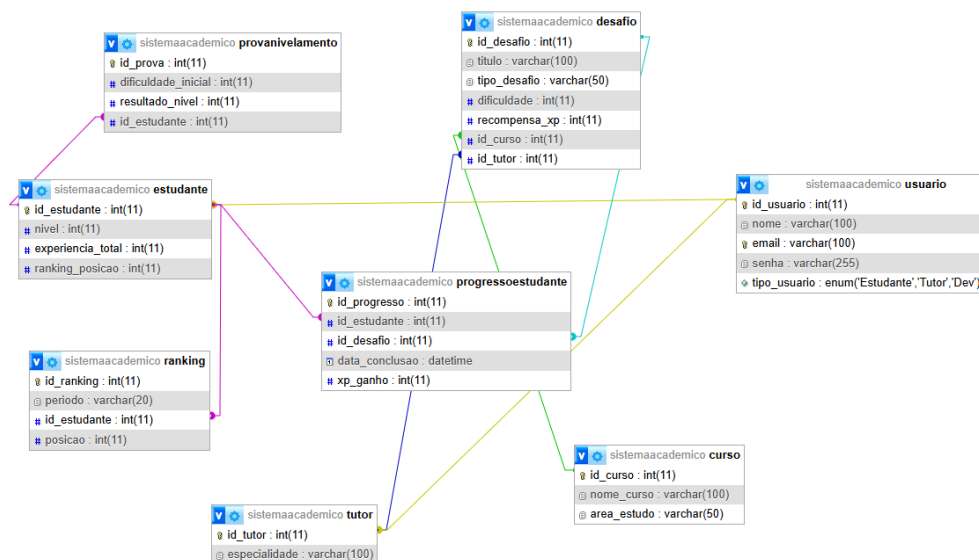


Figura 5 – DER – Diagrama Entidade-Relacionamento (SistemaAcademico)

2.2.3 MER – Modelo Entidade-Relacionamento

O MER a seguir detalha as entidades, atributos e relacionamentos com suas cardinalidades:

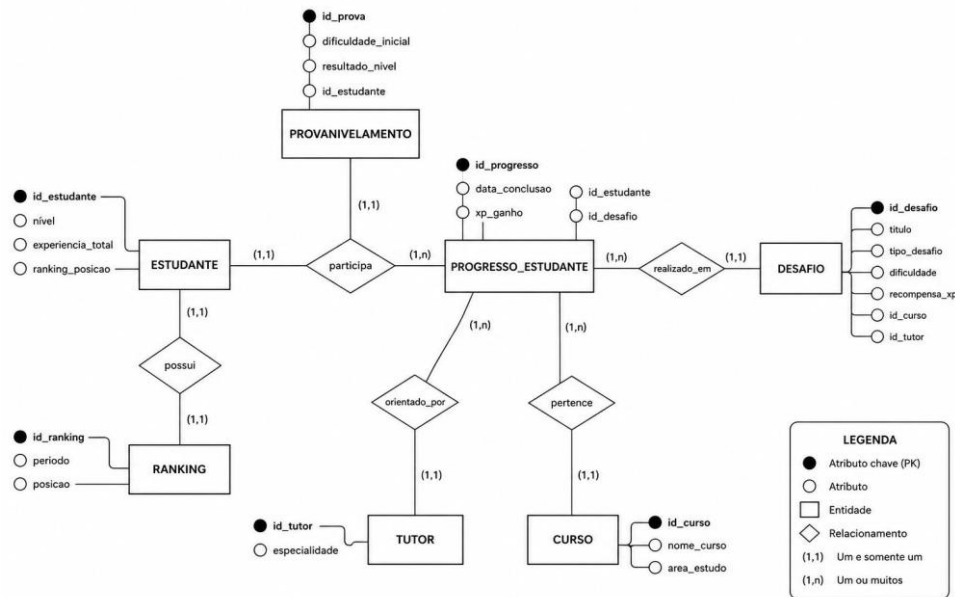


Figura 6 – MER – Modelo Entidade-Relacionamento

2.2.4 Entidades, Atributos, Relacionamentos e Cardinalidades

As entidades e seus atributos são:

- Usuário: id_usuario (PK), nome, email (UNIQUE), senha, tipo_usuario (ENUM).
- Estudante (especialização): id_estudante (PK/FK), nivel, experiencia_total, ranking_posicao.
- Tutor (especialização): id_tutor (PK/FK), especialidade.
- Curso: id_curso (PK), nome_curso, area_estudo.
- Desafio: id_desafio (PK), titulo, tipo_desafio, dificuldade, recompensa_xp, id_curso (FK), id_tutor (FK).
- ProgressoEstudante: id_progresso (PK), id_estudante (FK), id_desafio (FK), data_conclusao, xp_ganho.
- Ranking: id_ranking (PK), periodo, id_estudante (FK), posicao.

Relacionamentos e cardinalidades:

- Usuário possui Perfil (1:1): um usuário pode ser Estudante ou Tutor.
- Tutor cria Desafio (1:N): um tutor cria múltiplos desafios.
- Curso engloba Desafios (1:N): um curso contém vários desafios.
- Estudante realiza Prova de Nivelamento (1:N): um estudante pode ter vários registros de provas.

- Estudante completa Desafio (N:N): relação resolvida pela tabela ProgressoEstudante.
- Estudante figura no Ranking (1:N): um estudante pode aparecer em diferentes períodos.

A exclusão em cascata (ON DELETE CASCADE) garante integridade referencial ao remover um Usuário e seus perfis vinculados. A unicidade do campo email previne duplicidade de contas.

2.3 Diagramas UML

2.3.1 Diagrama de Casos de Uso

O diagrama de casos de uso apresenta os três atores do sistema (Estudante, Tutor e Dev/Administrador) e suas respectivas interações com as funcionalidades da plataforma.

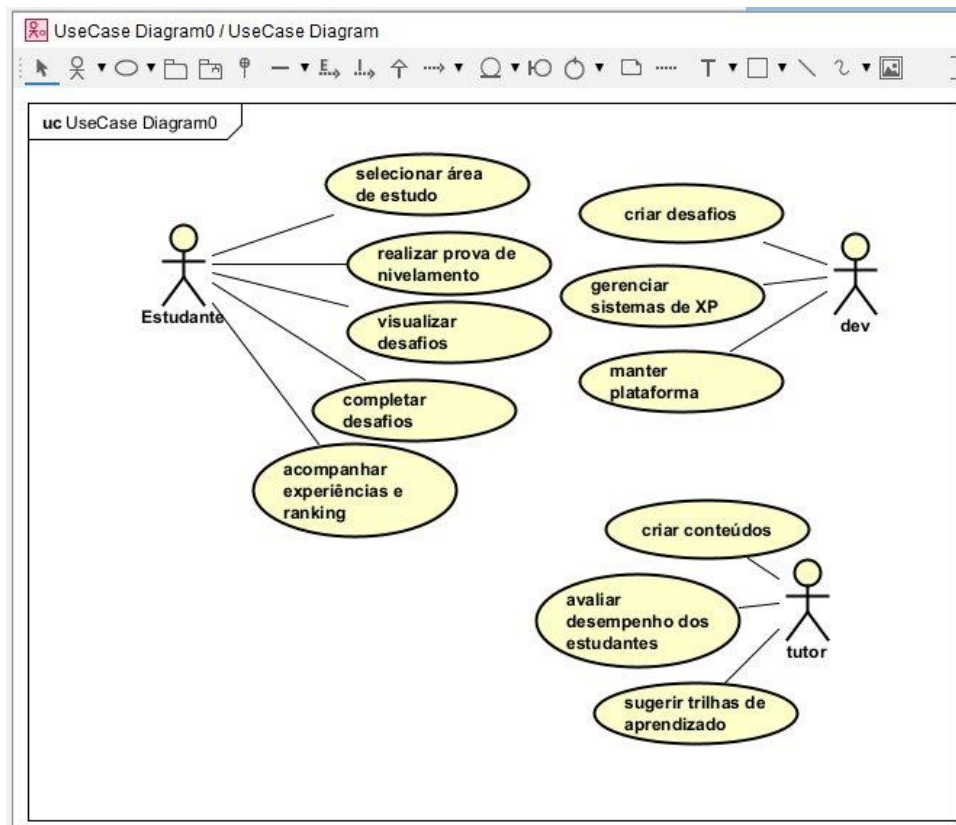


Figura 1 – Diagrama de Casos de Uso (UseCase Diagram)

O Estudante pode: selecionar área de estudo, realizar prova de nivelamento, visualizar e completar desafios, e acompanhar experiências e ranking. O Tutor pode: criar conteúdos, avaliar desempenho dos estudantes e sugerir trilhas de aprendizado. O Dev/Administrador pode: criar desafios, gerenciar sistemas de XP e manter a plataforma.

2.3.2 Diagramas de Classes

Os diagramas de classes representam a estrutura orientada a objetos do sistema, com as classes, atributos, métodos e seus relacionamentos.

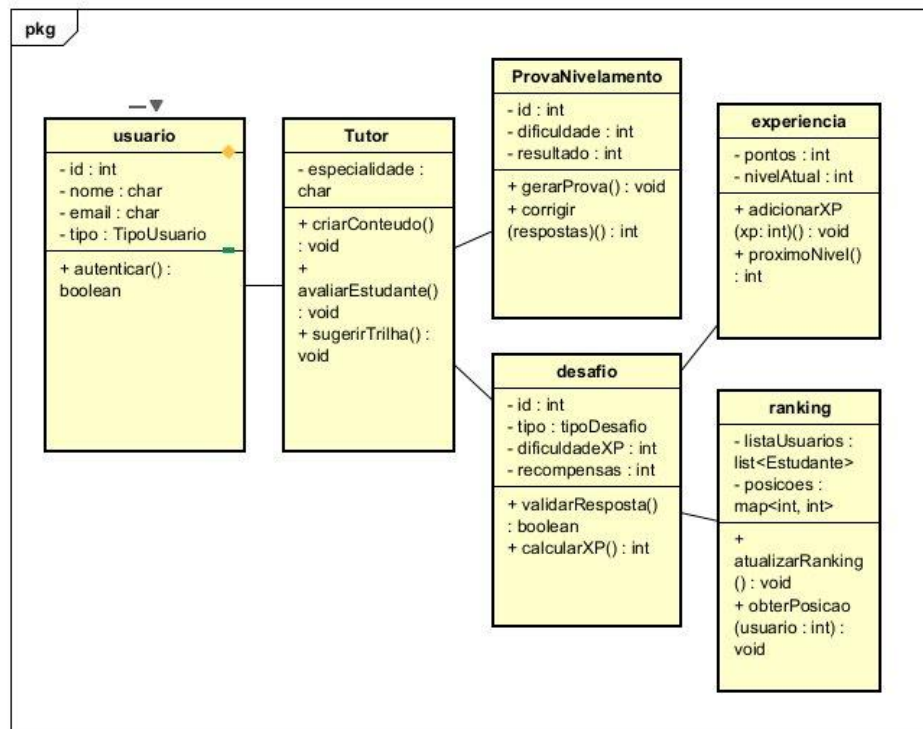


Figura 2 – Diagrama de Classes: visão geral (Usuário, Tutor, ProvaNivelamento, Desafio, Experiência e Ranking)

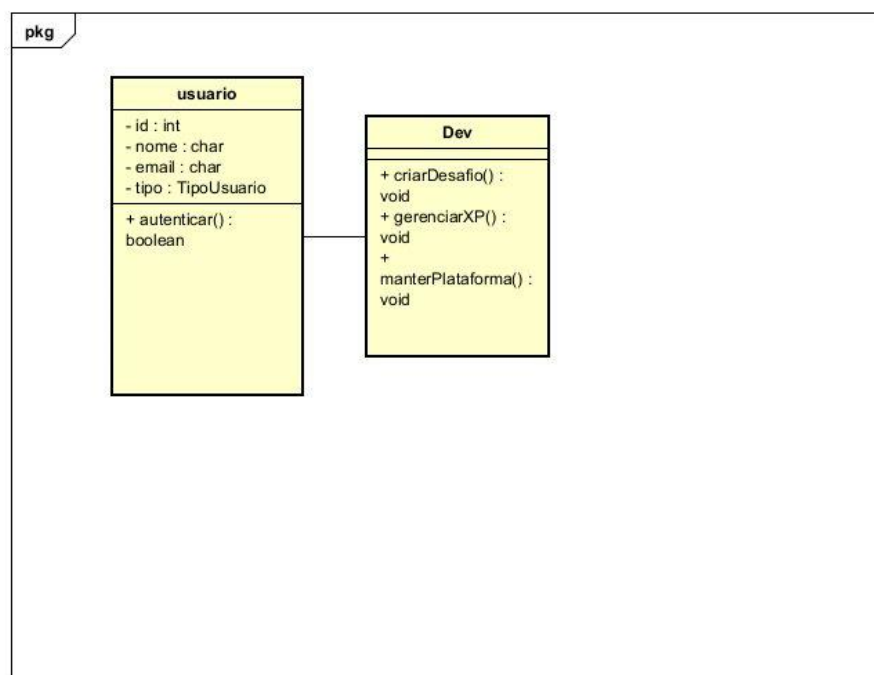


Figura 3 – Diagrama de Classes: Usuário e Dev

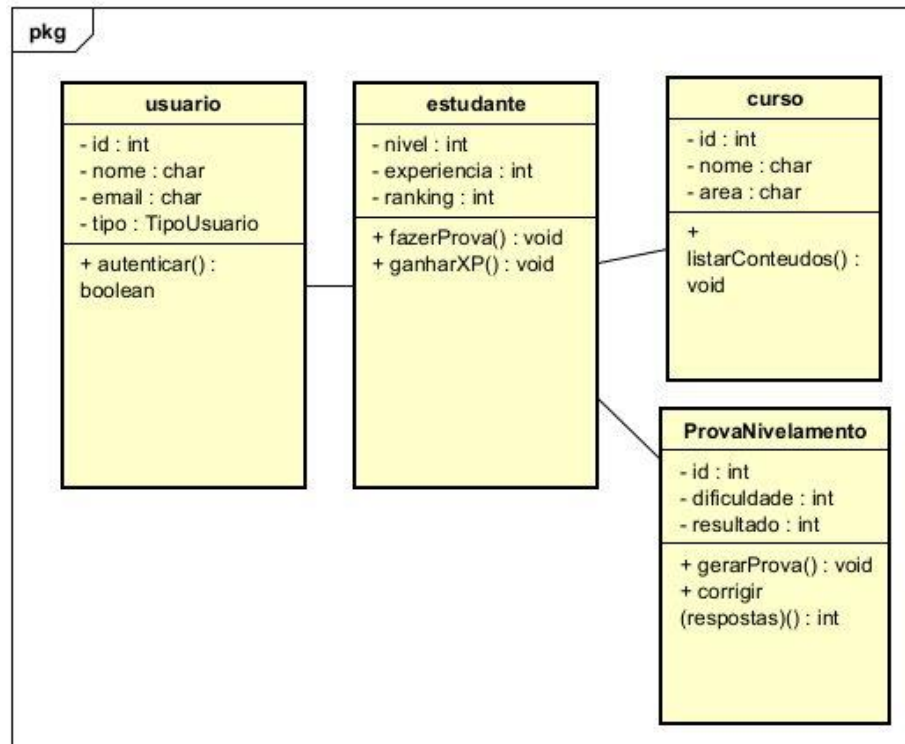


Figura 4 – Diagrama de Classes: Usuário, Estudante, Curso e ProvaNivelamento

As principais classes do sistema são:

- Usuario (classe abstrata): id, nome, email, tipo; método autenticar().
- Estudante (herda de Usuario): nivel, experiencia, ranking; métodos ganharXP() e fazerProva().
- Tutor (herda de Usuario): especialidade; métodos criarConteudo(), avaliarEstudante() e sugerirTrilha().
- Dev/Administrador (herda de Usuario): criarDesafio(), gerenciarXP() e manterPlataforma().
- Desafio: id, tipo, dificuldadeXP; métodos validarResposta() e calcularXP().
- ProvaNivelamento: id, dificuldade; métodos gerarProva() e corrigir(respostas).
- Ranking: listaUsuarios, posicoes; métodos atualizarRanking() e obterPosicao().

2.3.3 Diagramas de Sequência

Os diagramas de sequência descrevem o fluxo de interações entre os objetos ao longo do tempo.

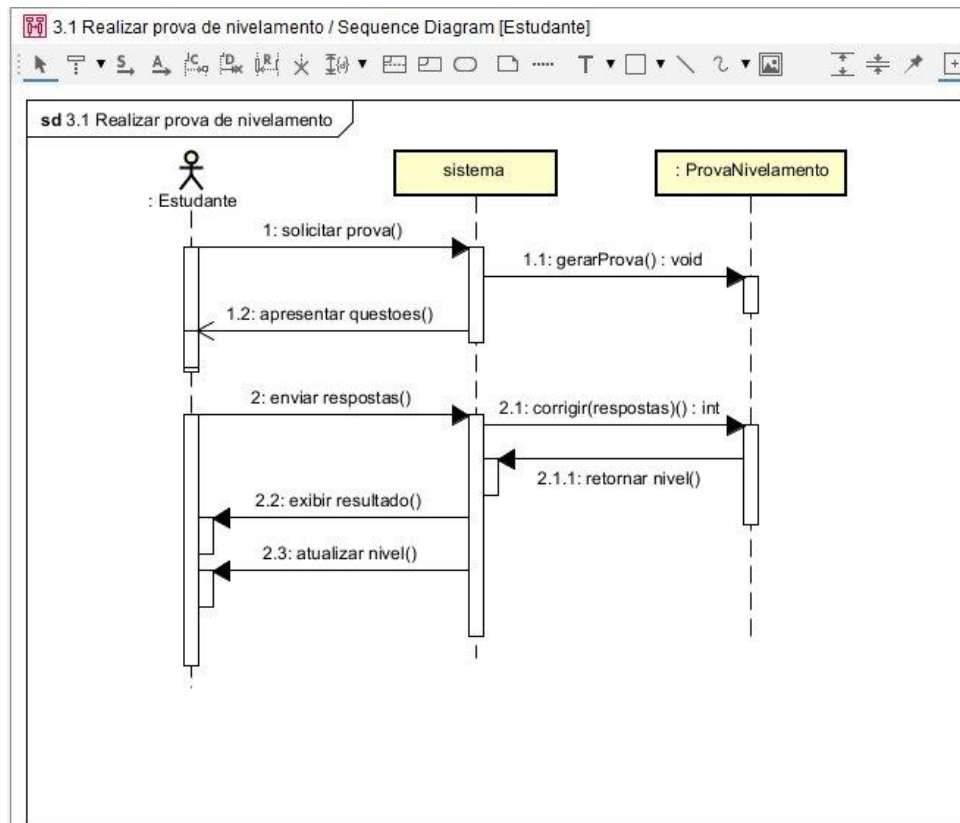


Figura 7 – Diagrama de Sequência: Realizar Prova de Nivelamento

No fluxo de realização da prova de nivelamento, o Estudante solicita a prova ao sistema, que aciona o objeto ProvaNivelamento para gerar e apresentar as questões. Após o envio das respostas, o sistema corrige e retorna o nível calculado, atualizando o perfil do estudante.

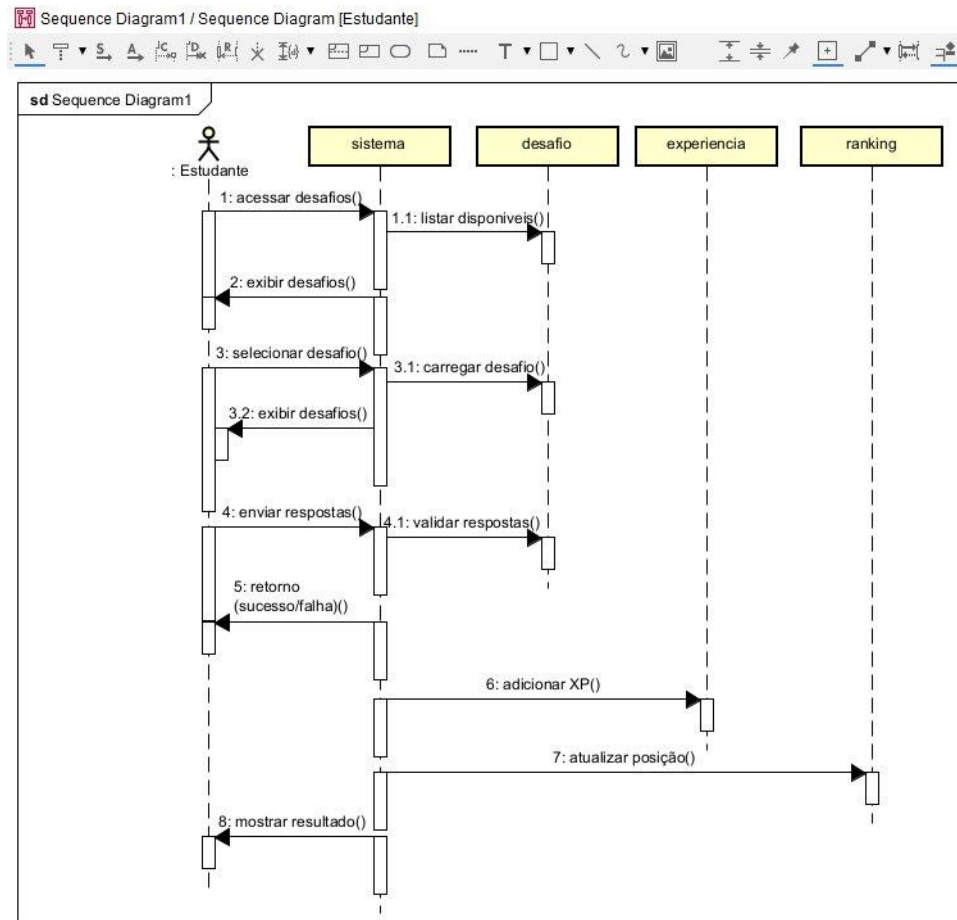


Figura 8 – Diagrama de Sequência: Completar Desafio

No fluxo de completar desafio, o Estudante acessa os desafios disponíveis, seleciona um, recebe as questões e envia as respostas. O sistema valida as respostas junto ao objeto Desafio; em caso de acerto, o XP é adicionado ao perfil do estudante e o ranking é atualizado automaticamente.

2.4 Back-End em C# – Programação Orientada a Objetos

O back-end do DojoDev foi implementado em C# utilizando os quatro pilares da Programação Orientada a Objetos (POO): abstração, encapsulamento, herança e polimorfismo.

2.4.1 Estrutura de Classes e Abstração

A classe abstrata `Usuario` define a estrutura base para todos os perfis do sistema. As classes `Estudante`, `Tutor` e `Dev` herdam de `Usuario` e implementam comportamentos específicos, aplicando polimorfismo através do método abstrato `Autenticar()`:

```
public abstract class Usuario
{
    public int Id { get; set; }
    public string Nome { get; set; }
    public string Email { get; set; }
    public TipoUsuario Tipo { get; set; }

    public Usuario(string nome, string email, TipoUsuario tipo)
    { Nome = nome; Email = email; Tipo = tipo; }

    public abstract bool Autenticar(string senha);
}
```

2.4.2 Herança e Encapsulamento – Classe Estudante

A classe `Estudante` demonstra herança (herda de `Usuario`) e encapsulamento (`NivelAtual` e `Experiencia` são somente leitura externamente):

```
public class Estudante : Usuario
{
    public int NivelAtual { get; private set; }
    public int Experiencia { get; private set; }
    public int ProximoNivel { get; private set; }

    public void GanharXP(int xp)
    { Experiencia += xp; AtualizarNivel(); }

    private void AtualizarNivel()
    {
        while (Experiencia >= ProximoNivel) {
            Experiencia -= ProximoNivel;
            NivelAtual++;
            ProximoNivel = NivelAtual * 100;
            Console.WriteLine($"{Nome} subiu para o nivel {NivelAtual}!");
        }
    }
}
```

2.4.3 Classe Desafio e Validação de Respostas

A classe `Desafio` encapsula título, tipo, dificuldade e XP de recompensa. O método `ValidarRespostas` compara as respostas fornecidas com as corretas:

```
public class Desafio
```

```
{
    public string Titulo { get; set; }
    public int DificuldadeXP { get; set; }
    private List<string> respostasCorretas = new List<string>();

    public bool ValidarRespostas(List<string> respostas)
    {
        for (int i = 0; i < respostas.Count; i++)
            if (!respostas[i].Equals(respostasCorretas[i],
                StringComparison.OrdinalIgnoreCase)) return false;
        return true;
    }
}
```

2.4.4 Classe Ranking

A classe Ranking mantém a lista de estudantes ordenada por XP e fornece a posição de cada um:

```
public class Ranking
{
    public List<Estudante> Posicoes { get; private set; } = new();

    public void Atualizar()
    { Posicoes.Sort((a, b) => b.Experiencia.CompareTo(a.Experiencia)); }

    public int ObterPosicao(Estudante e) => Posicoes.IndexOf(e) + 1;
}
```

A aplicação dos conceitos de POO garante baixo acoplamento e alta coesão entre as classes, facilitando a manutenção e evolução do sistema.

2.5 Desenvolvimento Web Responsivo

O front-end do DojoDev foi construído inteiramente com tecnologias web padrão: HTML5 semântico, CSS3 (Flexbox, Grid e Media Queries) e JavaScript Vanilla, sem dependência de frameworks de renderização.

2.5.1 Estrutura de Arquivos

O projeto contém as seguintes páginas HTML principais:

- index.html – página de cadastro de novos usuários;
- login.html – tela de autenticação com seleção de perfil (Aluno, Professor, Admin);
- dojo.html – dashboard principal do estudante com XP, nível e streak;
- jornada.html – trilhas de aprendizado com quizzes e desafios de código;
- missoes.html – página de missões e desafios disponíveis;
- professor.html – painel do tutor com criação de trilhas e acompanhamento de alunos;
- admin.html – dashboard administrativo com métricas e gerenciamento;
- perfil.html – perfil do estudante com XP e histórico;
- nivel.html / faixa.html – páginas de progresso e classificações;
- sensei.html – chat assistente interativo.

2.5.2 Responsividade

O layout foi projetado com breakpoints para diferentes dispositivos:

- Desktop: 1200px ou mais;
- Tablet: entre 768px e 1199px;
- Mobile: até 767px;
- Mobile pequeno: até 480px;
- Modo paisagem: otimizado com media queries específicas.

2.5.3 Sistema de Design

O design segue uma paleta dark com variáveis CSS globais:

```
:root {
  --bg:      #000000;    /* Fundo preto */
  --surface: #14141e;    /* Superfícies */
  --purple:  #7c6af0;    /* Roxo principal */
  --green:   #b4ff3e;    /* Verde neon */
}
```

```
--red:      #ff4d4d;    /* Vermelho */  
}
```

A consistência visual é garantida pelo arquivo style.css centralizado, com mais de 900 linhas de regras que cobrem componentes reutilizáveis como cards, botões, barras de progresso e navegação.

2.6 UX e UI Design

2.6.1 Personas

Foram definidas três personas representativas dos usuários do sistema:

- João, 19 anos, estudante ingressante em programação. Busca aprender de forma lúdica e motivacional. Acessa principalmente pelo celular.
- Ana, 28 anos, tutora e desenvolvedora front-end. Deseja criar conteúdos orgânicos e acompanhar o desempenho dos alunos de forma ágil.
- Carlos, 35 anos, administrador do sistema. Precisa de visão geral das métricas e controle de usuários com mínimo de esforço operacional.

2.6.2 Fluxos de Navegação

O fluxo principal do estudante segue a sequência: Cadastro → Login → Dashboard → Jornada (trilhas) → Desafio → Resultado + XP → Ranking. O fluxo do professor é: Login → Painel Professor → Criar Trilha → Gerenciar Alunos.

2.6.3 Princípios de Usabilidade Aplicados

As interfaces foram projetadas com foco em usabilidade, clareza visual e facilidade de uso:

- Hierarquia visual clara: informações mais importantes com maior destaque (XP, nível, streak);
- Feedback imediato: notificações toast ao completar ações;
- Navegação intuitiva: menu fixo com ícones reconhecíveis;
- Cores significativas: verde para sucesso/progresso, vermelho para erros/alertas, roxo para elementos neutros;
- Acessibilidade básica: contraste adequado e áreas de toque generosas para mobile.

2.7 Machine Learning e Análise de Dados

Esta etapa descreve como o DojoDev pode utilizar dados gerados pela plataforma para aprimorar a experiência de aprendizado e apoiar a tomada de decisão pedagógica.

2.7.1 Dados Relevantes para Análise

A plataforma gera continuamente dados valiosos que podem ser explorados analiticamente:

- XP acumulado por estudante ao longo do tempo;
- Taxa de conclusão de desafios por nível de dificuldade;
- Tempo médio de resposta em provas de nivelamento;
- Padrões de acesso (horários, frequência, streaks);
- Desafios com maior taxa de erro (indicadores de conteúdo difícil).

2.7.2 Técnicas de Análise de Dados Aplicáveis

Com base nos dados coletados, as seguintes técnicas podem ser aplicadas:

- Análise descritiva: cálculo de médias, medianas e percentis de XP por trilha, identificando os pontos de maior dificuldade;
- Classificação supervisionada (k-NN ou árvore de decisão): predição do nível ideal para novos estudantes com base em respostas da prova de nivelamento;
- Clusterização (k-Means): agrupamento de estudantes por perfil de desempenho, permitindo sugestões de trilhas personalizadas;
- Análise de séries temporais: identificação de padrões de engajamento ao longo das semanas.

2.7.3 Indicadores e Relatórios

O painel administrativo pode exibir os seguintes indicadores gerados pela análise:

- Taxa de retenção semanal de estudantes ativos;
- Ranking dinâmico por trilha e por período;
- Alertas automáticos para estudantes com queda de desempenho;

- Relatório de desafios mais e menos concluídos, auxiliando tutores na revisão de conteúdo.

A aplicação consciente dessas técnicas transforma o DojoDev em uma plataforma adaptativa, capaz de personalizar a experiência de cada estudante com base em dados reais de uso.

2.8 Comunicação, Liderança, Negociação e LIBRAS

2.8.1 Estratégias de Comunicação no Projeto

O desenvolvimento do DojoDev exigiu comunicação estruturada entre os seis integrantes do grupo. As principais estratégias adotadas foram:

- Reuniões semanais de alinhamento com pauta definida previamente, garantindo objetividade e registro de decisões;
- Uso de canal de mensagens instantâneas para comunicação rápida e resolução de bloqueios técnicos;
- Documentação contínua do progresso, com cada membro responsável por registrar suas entregas;
- Feedback construtivo entre os integrantes ao revisar código e interfaces, fortalecendo a qualidade do produto final.

2.8.2 Situações de Liderança e Negociação

Ao longo do projeto, diversas situações exigiram habilidades de liderança e negociação:

- Definição de prioridades: quando surgiram demandas conflitantes entre funcionalidades (ex.: priorizar responsividade mobile vs. novas telas), o grupo negociou com base em critérios técnicos e no impacto para o usuário final;
- Divisão de tarefas: o líder técnico distribuiu responsabilidades conforme as competências individuais, equilibrando carga de trabalho e evitando gargalos;
- Resolução de conflitos técnicos: divergências sobre arquitetura do banco de dados e nomenclatura de classes foram resolvidas por consenso, com argumentação técnica embasada nas normas estudadas.

2.8.3 Proposta de Acessibilidade com LIBRAS

Em atenção aos princípios de inclusão e acessibilidade, o grupo propõe as seguintes ações para incorporar LIBRAS ao DojoDev:

- Legendas e transcrições: todas as videoaulas e conteúdos multimídia da plataforma deverão ter transcrição textual e opção de legenda, facilitando o acesso para usuários surdos;

- Botão de acessibilidade: inclusão de um botão fixo na interface que ativa um modo de acessibilidade, aumentando contraste e ativando leitores de tela compatíveis com tecnologias assistivas;
- Glossário em LIBRAS: os termos técnicos mais utilizados na plataforma (ex.: variável, loop, função) serão apresentados com vídeos curtos em LIBRAS, produzidos em parceria com integrantes ou colaboradores da comunidade surda;
- Documentação acessível: este próprio trabalho acadêmico adota linguagem clara e objetiva, contribuindo para a acessibilidade cognitiva do conteúdo.

Essas iniciativas alinham o DojoDev às diretrizes da Lei Brasileira de Inclusão (Lei nº 13.146/2015) e ao Decreto nº 5.626/2005, que regulamenta o uso de LIBRAS como primeira língua para pessoas surdas no Brasil.

2.9 Integração do Sistema

O DojoDev integra as camadas desenvolvidas em um sistema coerente e funcional. A lógica de negócio definida nas classes C# corresponde diretamente às interações do usuário nas interfaces web.

A arquitetura geral do sistema é descrita em três camadas:

- Camada de Apresentação: HTML5 + CSS3 + JavaScript Vanilla, com design responsivo e dark theme;
- Camada de Lógica de Negócio: classes C# implementando POO (herança, encapsulamento, polimorfismo);
- Camada de Dados: banco de dados MySQL relacional com 7 tabelas interligadas.

As limitações atuais incluem o uso de localStorage para persistência no front-end, em substituição a um back-end integrado com banco de dados real. Como proposta de evolução, o sistema poderia ser migrado para uma arquitetura cliente-servidor com API REST em C# (ASP.NET Core) conectada ao MySQL.

3 CONCLUSÃO

O desenvolvimento do DojoDev demonstrou de forma prática a integração dos conhecimentos adquiridos no terceiro semestre do curso de Análise e Desenvolvimento de Sistemas da UNIP. As disciplinas de Modelagem de Banco de Dados, Programação Orientada a Objetos com C#, Desenvolvimento Web Responsivo, UX/UI Design, Machine Learning e Análise de Dados e Comunicação e Liderança foram aplicadas de forma harmônica na construção de uma plataforma funcional.

Os objetivos propostos foram atingidos: o sistema oferece trilhas de aprendizado gamificadas, mecanismo de pontuação por XP, prova de nivelamento, ranking, painel administrativo e proposta concreta de acessibilidade com LIBRAS. A arquitetura orientada a objetos garante extensibilidade do código, e o design responsivo assegura acessibilidade em diferentes dispositivos.

Entre as principais aprendizagens do grupo destacam-se: a importância do planejamento com diagramas UML antes da implementação, a necessidade de normalização do banco de dados para evitar redundâncias, o valor da consistência visual na experiência do usuário e a relevância da comunicação estruturada para o sucesso de projetos em equipe.

Como perspectivas de evolução futura, o projeto poderia incorporar: autenticação real com JWT, API REST em ASP.NET Core, sistema de rankings em tempo real, certificações digitais ao concluir trilhas, integração com IA real para o assistente Sensei, implementação efetiva dos recursos de LIBRAS e aplicação de modelos de machine learning para personalização da aprendizagem.

Conclui-se que o DojoDev representa um produto com potencial real de aplicação, unindo tecnologia e educação de forma inovadora, motivacional e inclusiva.

REFERÊNCIAS

DEITEL, Harvey M.; DEITEL, Paul J. C#: como programar. 6. ed. São Paulo: Pearson, 2011.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de banco de dados. 7. ed. São Paulo: Pearson, 2019.

SOMMERVILLE, Ian. Engenharia de software. 10. ed. São Paulo: Pearson, 2019.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

FOWLER, Martin. UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2005.

PROVDANOV, Cleber Cristiano; FREITAS, Ernani Cesar de. Metodologia do trabalho científico: métodos e técnicas da pesquisa e do trabalho acadêmico. 2. ed. Novo Hamburgo: Feevale, 2013.

MOZILLA DEVELOPER NETWORK. HTML: Hypertext Markup Language. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTML>. Acesso em: 20 maio 2026.

MOZILLA DEVELOPER NETWORK. CSS: Cascading Style Sheets. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/CSS>. Acesso em: 20 maio 2026.

TAILWIND CSS. Documentation. Disponível em: <https://tailwindcss.com/docs>. Acesso em: 20 maio 2026.

MYSQL. MySQL 8.0 Reference Manual. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: 20 maio 2026.

BRASIL. Lei nº 13.146, de 6 de julho de 2015. Lei Brasileira de Inclusão da Pessoa com Deficiência. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2015/lei/l13146.htm. Acesso em: 20 maio 2026.

BRASIL. Decreto nº 5.626, de 22 de dezembro de 2005. Regulamenta a Lei nº 10.436/2002 (LIBRAS). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2004-2006/2005/decreto/d5626.htm. Acesso em: 20 maio 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.